

Session 3

Pós-Graduação em Sistemas Computacionais Embebidos (PGSCE) / SwitCH Embedded

Desenvolvimento de Sistemas Embebidos (PGSCE-DSISEM) / Embedded Systems Development - 2025/2026

Jorge Estrela da Silva

1



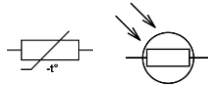
ANALOG TO DIGITAL CONVERTER

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

2

ANALOG SIGNALS AND DIGITAL SIGNALS

- **Analog signal:** continuous-time function of real values ($\mathbb{R} \rightarrow \mathbb{R}$).
 - Represents the variation in time of some physical quantity (temperature, pressure, velocity, etc).
 - For instance: the voltage level at the terminals of a thermistor or of a Light Dependent Resistor (LDR).



- **Digital signal:** discrete-time/sequence of quantized values (e.g., 8 bits)
 - Sequence of samples, encoded with a finite set of values/codes

Porto
TechHub

isep

Instituto Superior de
Engenharia do Porto

FEUP

Santander

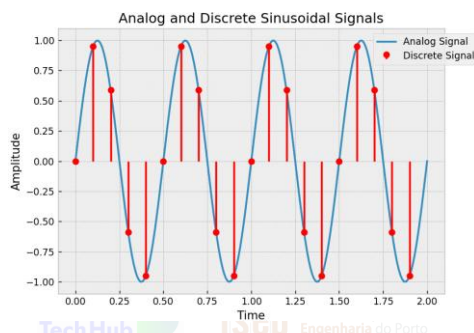
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

3

3

ANALOG, DISCRETE AND DIGITAL SIGNALS

Discrete-time sampling



TechHub

isep

Instituto Superior de
Engenharia do Porto

FEUP

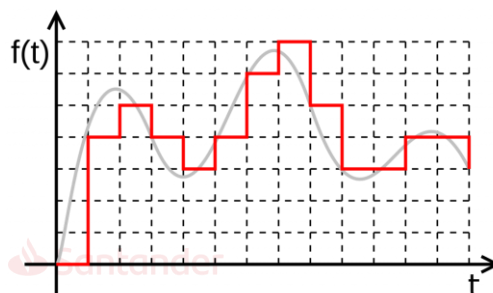
Santander

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

Digital signal:

Discrete-time + quantization

- In this case, zero-order hold reconstruction of signal is employed.



4

4

ANALOG TO DIGITAL CONVERTER

- Analog signal: continuous-time function of real values ($\mathbb{R} \rightarrow \mathbb{R}$).
 - Represents the variation in time of some physical quantity (temperature, pressure, velocity, etc).
 - For instance: the voltage level at the terminals of a thermistor or of a Light Dependent Resistor (LDR).
- Digital signal: discrete-time/sequence of quantized values (e.g., 8 bits)
- An analog to digital converter (ADC) generates a quantized sample of the current value of the analog signal at its input.
 - Sample and hold ensures input voltage is held constant during conversion



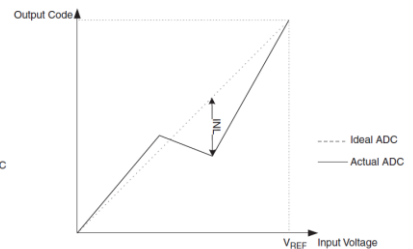
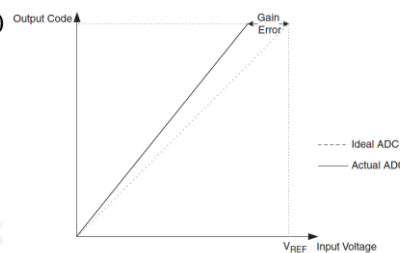
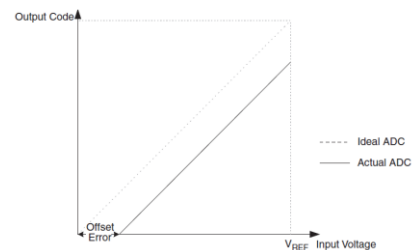
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

5

5

RESOLUTION VS ACCURACY

- Resolution dictated by number of bits
- Accuracy depends on:
 - Measure Variance (as close to zero as possible)
 - Offset (should be 0)
 - Gain error (should be 0)
 - Linearity



Porto TechHub isep Institute Engenharia

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

6

6

SAMPLING RATE

- Should be suited for the process/system being monitored
 - Room heating system - a few readings per hour
 - Motor control – ~1..100 kHz
- In some cases, it is possible to establish a tradeoff between sampling rate and accuracy
 - For instance, the ADC from Atmega128 (successive approximations)



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

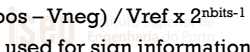


7

ANALOG TO DIGITAL CONVERTER

- Two typical input options:
 - Single ended input:
 - $ADC = V_{in}/V_{ref} \times 2^{n_{bits}}$
 - Value of LSB = $V_{ref} / 2^{n_{bits}}$
 - Example for 4-bit ADC, with $V_{ref} = 5\text{ V}$
 - $LSB = 5/2^4 = 0.3125\text{ V}$
 - Input from 0 to 0.5 LSB => ADC output = 0
 - Input from 0.5 LSB to 1.5 LSB => ADC output = 1
 - Input greater than $V_{ref} - 1.5\text{ LSB}$ => ADC output = 0xF
 - Differential mode: the output value is the difference between the values at two inputs
 - $ADC = (V_{pos} - V_{neg}) / V_{ref} \times 2^{n_{bits}-1}$
 - One bit used for sign information

VREF = 5		
ADC Input Interval (V)		ADC output
0	0,15625	0
0,15625	0,46875	1
0,46875	0,78125	2
0,78125	1,09375	3
1,09375	1,40625	4
1,40625	1,71875	5
1,71875	2,03125	6
2,03125	2,34375	7
2,34375	2,65625	8
2,65625	2,96875	9
2,96875	3,28125	10
3,28125	3,59375	11
3,59375	3,90625	12
3,90625	4,21875	13
4,21875	4,53125	14
4,53125		15



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

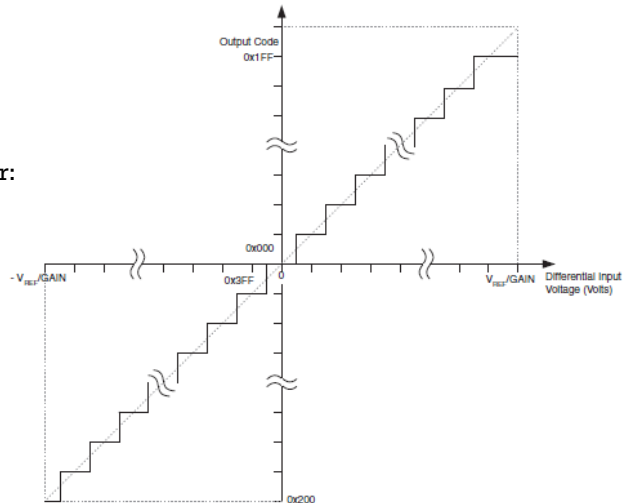


8

DIFFERENTIAL MEASUREMENT RANGE

- Example for 10 bit ADC
 - 0x200 to 0x1FF (two's complement)
 - Two's complement of negative number:
 - $2^{nbits} - (\text{absolute value})$

V_{ADCR}	Read code	decimal
$V_{ADCR} + V_{REF}/GAIN$	0x1FF	511
$V_{ADCR} + 511/512 V_{REF}/GAIN$	0x1FF	511
$V_{ADCR} + 511/512 V_{REF}/GAIN$	0x1FE	510
...	...	...
$V_{ADCR} + 1/512 V_{REF}/GAIN$	0x001	1
V_{ADCR}	0x000	0
$V_{ADCR} - 1/512 V_{REF}/GAIN$	0x3FF	-1
...	...	...
$V_{ADCR} - 511/512 V_{REF}/GAIN$	0x201	-511
$V_{ADCR} - V_{REF}/GAIN$	0x200	-512



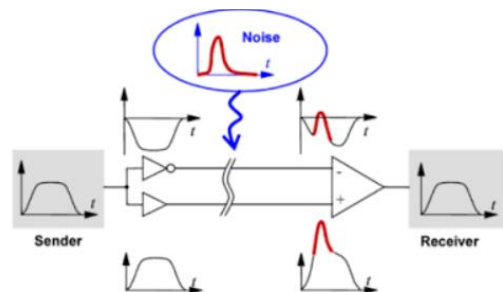
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

9

9

ANALOG TO DIGITAL CONVERTER

- Two typical modes of operations:
 - Single ended input
 - Differential mode:** the output value is the difference between the values at two inputs
 - $ADC = (V_{pos} - V_{neg}) / V_{ref} \times 2^{nbits-1}$
 - One bit used for sign information
 - One of the applications of differential mode is to increase noise rejection in signal transmission



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

10

10

ADMUX REGISTER

Bit	7	6	5	4	3	2	1	0	
	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Reference Voltage Selection - Bits 7:6 of ADMUX (REFS1:0)

REFS1	REFS0	Voltage Reference Selection
0	0	AREF, Internal Vref turned off
0	1	AVCC with external capacitor at AREF pin
1	0	Reserved
1	1	Internal 2.56V Voltage Reference with external capacitor at AREF pin



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

13

13

ADMUX REGISTER LEFT-ADJUSTED RESULT

- Bit 5 of ADMUX (ADLAR) – Affects how conversion result is stored in the data registers (ADCL and ADCH)

- ADLAR = 1 (Left Adjust the Result)

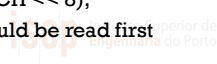
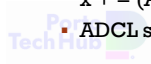
- If only 8-bit precision is required, read only ADCH (will contain 8 most significant bits)
- uint8_t x = ADCH;

- ADLAR = 0

- Standard memory organization
- uint16_t x = ADCL;
x += (ADCH << 8);
- ADCL should be read first

Bit	15	14	13	12	11	10	9	8	
	ADC9	ADC8	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADCH
	ADC1	ADC0	–	–	–	–	–	–	ADCL
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

Bit	15	14	13	12	11	10	9	8	
	–	–	–	–	–	–	ADC9	ADC8	ADCH
	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0	ADCL
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	



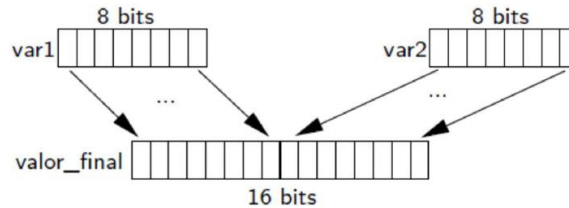
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

14

14

BYTE MANIPULATION

- `uint16_t valor_final = var2;`
`valor_final += (var1 << 8);`



- `ADLAR = 1`, 10 bit read:
 - `uint16_t x = ADCL`
`x += (ADCH << 8);`
`x = x >> 6; //one extra step`

15

Mux 4:0	Entrada Single Ended	Entrada Diferencial Positiva	Entrada Diferencial Negativa	Ganho
00000	ADC0	NA	NA	
00001	ADC1			
00010	ADC2			
00011	ADC3			
00100	ADC4			
00101	ADC5			
00110	ADC6			
00111	ADC7			
01000	NA	ADC0	ADC0	10x
01001		ADC1	ADC0	10x
01010		ADC0	ADC0	200x
01011		ADC1	ADC0	200x
01100		ADC2	ADC2	10x
01101		ADC3	ADC2	10x
01110		ADC2	ADC2	200x
01111		ADC3	ADC2	200x
10000		ADC0	ADC1	1x
10001		ADC1	ADC1	1x
10010		ADC2	ADC1	1x
10011		ADC3	ADC1	1x
10100		ADC4	ADC1	1x
10101		ADC5	ADC1	1x
10110		ADC6	ADC1	1x
10111		ADC7	ADC1	1x
11000		ADC0	ADC2	1x
11001		ADC1	ADC2	1x
11010		ADC2	ADC2	1x
11011		ADC3	ADC2	1x
11100		ADC4	ADC2	1x
11101		ADC5	ADC2	1x
11110	1.23V (V _{BG})	NA		
11111	0V (GND)	NA		

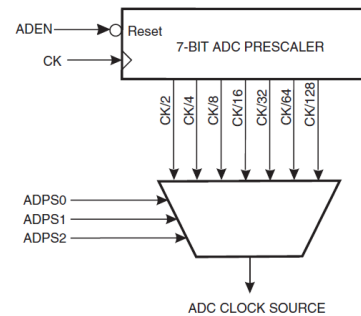
ADMUX REGISTER CHANNEL SELECTION

- Bits 4:0 of ADMUX (MUX4:0)
- Changes only take effect after end of conversion.
- 200x gain implies lower accuracy (7 MSB)
- In yellow: can be used to estimate offset in the analog path
 - Measured value can be subtracted from subsequent measures
 - Ideally should be 0.

16

CONVERSION TIMING

- ATmega128 ADC requires a clock frequency between 50 kHz and 200 kHz for optimal accuracy
- ADC contains a prescaler, which allows selection of different ADC clock frequencies.
- Higher ADC clock frequencies allow higher sampling rates
 - But lower accuracy



17

ADC CONTROL AND STATUS REGISTER

Bit	7	6	5	4	3	2	1	0	
	ADEN	ADSC	ADFR	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **ADPS2:1: Prescaler Selection**

ADPS2	ADPS1	ADPS0	Division Factor
0	0	0	2
0	0	1	2
0	1	0	4
0	1	1	8
1	0	0	16
1	0	1	32
1	1	0	64
1	1	1	128

18

CONVERSION TIMING

- First conversion always takes more time
 - 25 vs 13 ADC cycles/sample
- Example for 16 MHz CPU clock:
 - 1/64 prescaler: $16 \text{ MHz} / 64 = 667 \text{ kHz}$ ✗ (too high for 10 bit resolution to be meaningful)
 - 1/128 prescaler: $16 \text{ MHz} / 128 = 125 \text{ kHz}$ ✓ (below 200 kHz, suitable for 10 bit)
 - Maximum sampling rate after first conversion: $125 \text{ kHz} / 13 \text{ cycles/sample} = 9.6 \text{ kSPS}$
 - sample per second (SPS)
- ADC is optimized for a bandwidth of 4 kHz
 - Low pass filter should be used, if higher frequencies are expected

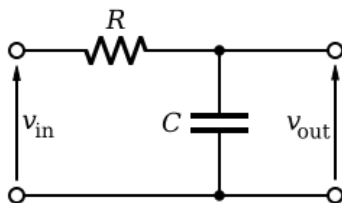
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

19

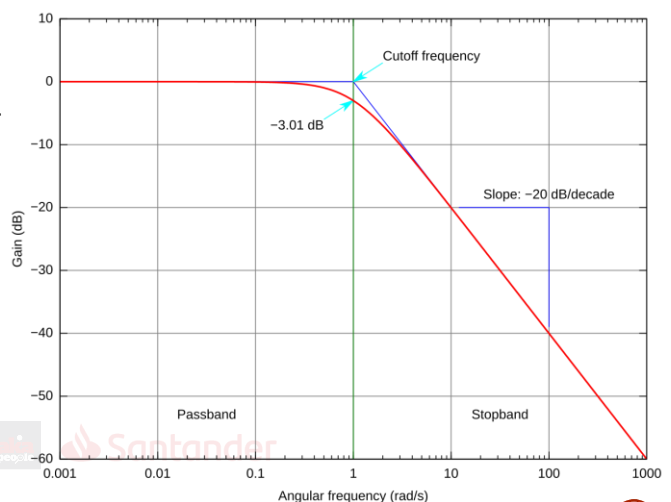
19

SIMPLEST LOW PASS FILTER

- First order RC filter
 - Frequency dependent voltage divider



- Cutoff frequency = $1/(2\pi RC)$



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

20

20

ADC CONTROL AND STATUS REGISTER

Bit	7	6	5	4	3	2	1	0	
	ADEN	ADSC	ADFR	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **ADEN**: set to 1 to **Enable AD**
- **ADSC**: set to 1 to **Start Conversion**
- **Two modes of operation**
 - **ADFR = 1** => “Free running”: the ADC starts a new conversion immediately after completing the previous one.
 - **ADFR = 0** => “One shot” conversion: ADC only starts a conversion after **ADSC** is set to 1
 - **ADSC** is automatically reset to 0 when the conversion is completed (but data registers might not yet be updated)

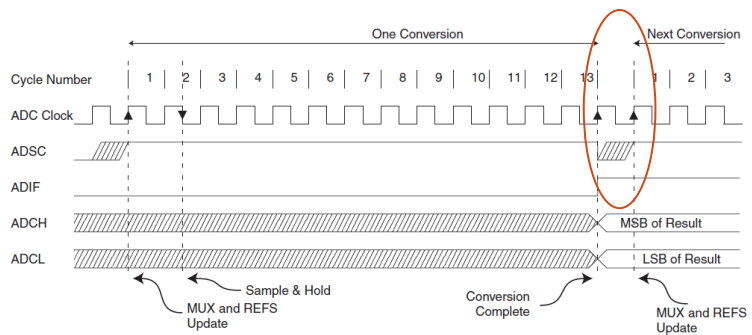
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

21

21

ADC MODES

- **Single Conversion**



- **Free Running**



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

22

22

ADC CONTROL AND STATUS REGISTER

Bit	7	6	5	4	3	2	1	0	
	ADEN	ADSC	ADFR	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **ADIF**: this bit is set when data registers are updated, after end of a conversion
 - If **ADIE** (AD Interrupt Enable) is 1, the ADC Conversion Complete Interrupt is triggered
 - Global Interrupt Enable must also be set.
 - If **ADIE** is 1, **ADIF** is automatically cleared when entering the ISR. Otherwise, it can be cleared by writing a 1 to it.



Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

23

23

EXERCISE

- Configure the ATmega128 ADC for the following scenario:
 - Single Ended Input, channel 2
 - No interrupt on conversion completion
 - Result justified to the right
 - “real” 10-bit resolution
 - Reference voltage AREF
 - CPU clock of 16 MHz
- Implement the `uint16_t ADC_read_channel2()` function that starts the conversion and returns the converted value.



Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

24

24

```

#include <avr/io.h>

void init() {
    ADMUX = 0b0000010; //AREF, RIGHT ADJUSTED, channel 2
    ADCSRA = 0b10000111; //ADEN, 1/128
}

uint16_t ADC_read_channel2() {
    uint8_t vh, vl;

    ADCSRA |= (1<<ADSC); //Start Conversion

    //wait for ADSC to go to 0 (signals end of conversion)
    while( ADCSR & (1<<ADSC) );

    vl = ADCL;
    vh = ADCH;

    return (vh << 8) + vl;
}

```



Instituto Superior de
Engenharia do Porto

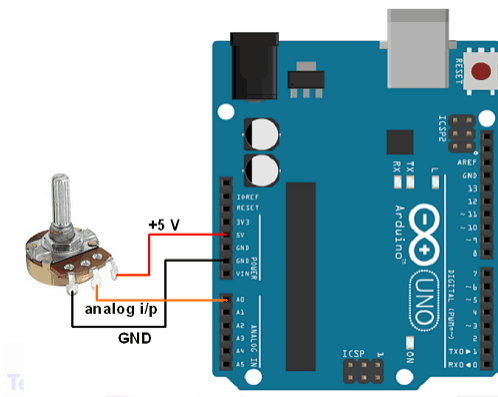


PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

25

25

A SIMPLE WAY TO TEST THE ADC OF THE ARDUINO



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

26

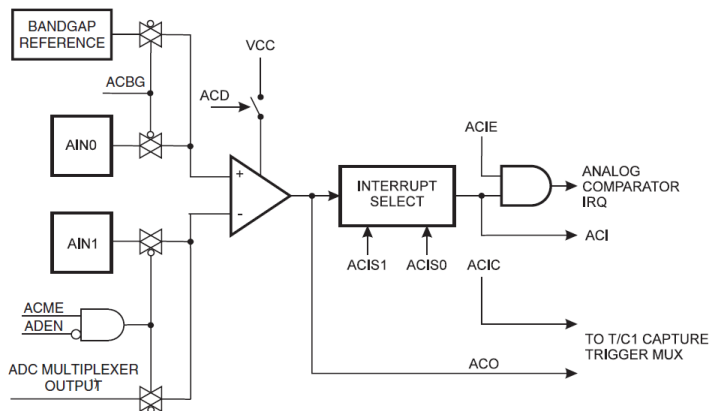
26



ANALOG COMPARATOR

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

27



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

ANALOG COMPARATOR

- $AIN0 > AIN1 \Rightarrow$ Analog Comparator Output (ACO) set to 1
- Can trigger interrupt on ACO toggle, rising edge or falling edge.



28

CONTROL/STATUS

Bit	7	6	5	4	3	2	1	0	
	ACD	ACBG	ACO	ACI	ACIE	ACIC	ACIS1	ACIS0	ACSR
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	N/A	0	0	0	0	0	

- Bit 7 – ACD: Analog Comparator Disable (to save power when not in use)
- Bit 6 – ACBG: Analog Comparator Bandgap Select:
 - If this bit is 1, AIN1 is compared with a fixed voltage (2.56 V) instead of AIN0
- Bit 5 – ACO: Analog Comparator Output (set to 1 if AIN0 > AIN1, 0 otherwise)
- Bit 2 – ACIC: Analog Comparator Input Capture Enable
 - ACO will be used to trigger Timer/Counter1 (will be seen later)



Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

29

29

CONTROL/STATUS - INTERRUPTS

Bit	7	6	5	4	3	2	1	0	
	ACD	ACBG	ACO	ACI	ACIE	ACIC	ACIS1	ACIS0	ACSR
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	N/A	0	0	0	0	0	

- Bits 1, 0 – ACIS1, ACIS0: Analog Comparator Interrupt Mode Select:

ACIS1	ACIS0	Interrupt Mode
0	0	Comparator Interrupt on Output Toggle
0	1	Reserved
1	0	Comparator Interrupt on Falling Output Edge
1	1	Comparator Interrupt on Rising Output Edge



Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

30

30

CONTROL/STATUS - INTERRUPTS

Bit	7	6	5	4	3	2	1	0	
	ACD	ACBG	ACO	ACI	ACIE	ACIC	ACIS1	ACIS0	ACSR
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	N/A	0	0	0	0	0	

- Bit 3 – ACIE: Analog Comparator Interrupt Enable
 - Set to 1 to activate interrupt
- Bit 4 – ACI: Analog Comparator Interrupt Flag. As usual:
 - Set to 1 when Comparator Output meets interrupt condition
 - Set to 0 when entering the ISR
 - If not using interrupts, cleared by writing 1 to it



Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

31

31

CONTROL/STATUS

- Bit 3 (ACME) of SFIOR: Analog Comparator Multiplexer Enable
 - If ACME is 1 and the ADC is switched off (ADEN in ADCSRA must be 0), the currently selected ADC channel is used instead of AIN1

ACME	ADEN	MUX2..0	Analog Comparator Negative Input
0	x	xxx	AIN1
1	1	xxx	AIN1
1	0	000	ADC0
1	0	001	ADC1
1	0	010	ADC2
1	0	011	ADC3
1	0	100	ADC4
1	0	101	ADC5
1	0	110	ADC6
1	0	111	ADC7



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

32

32

Session 4

Pós-Graduação em Sistemas Computacionais Embebidos (PGSCE) / SwitCH Embedded

Desenvolvimento de Sistemas Embebidos (PGSCE-DSISEM) / Embedded Systems Development - 2025/2026

Jorge Estrela da Silva

33



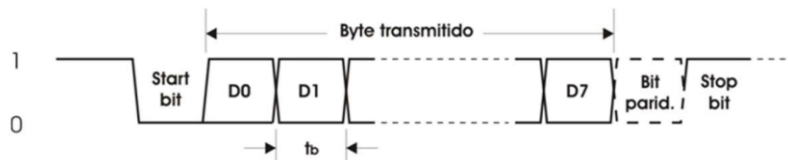
UNIVERSAL SYNCHRONOUS AND ASYNCHRONOUS SERIAL RECEIVER AND TRANSMITTER (USART)

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

34

SERIAL COMMUNICATION

- One bit transmitted at a time
 - Bit duration depends on the type of communication
- Example:



Instituto Superior de
Engenharia do Porto



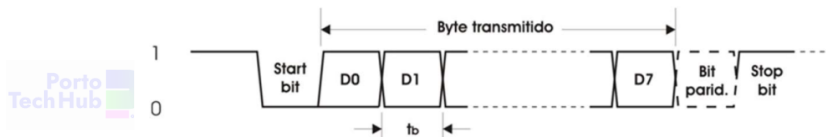
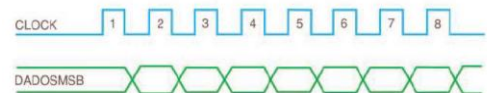
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

35

35

ASYNCHRONOUS AND SYNCHRONOUS SERIAL COMMUNICATION

- Synchronous transmission**
 - Includes a dedicated line for clock transmission
 - Transmission starts in synchronization with the clock
- Asynchronous communication:**
 - No dedicated line for clock
 - Transmission may start at any time
 - Requires special care with frame support so that clock information can be recovered
 - Start and stop bits



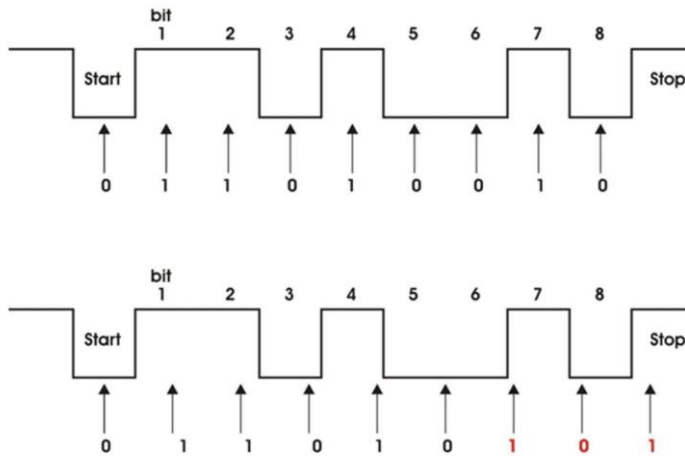
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

36

36

CLOCK DRIFT

- Emitter and Receptor clocks may have slightly different rates
- Timing errors accumulate at each bit



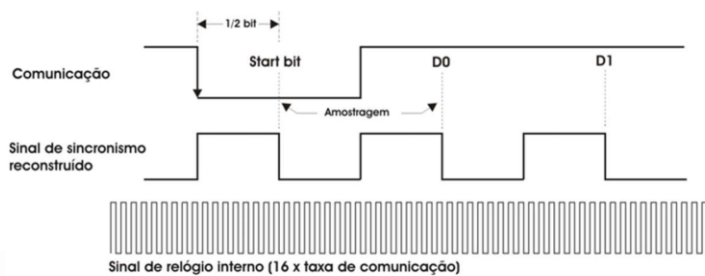
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

37

37

ASYNCHRONOUS COMMUNICATION

- Bit frame detection
 - Bit level is decided by "majority voting"



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

38

38

ASYNCHRONOUS COMMUNICATION

- Connection allowing full-duplex communication



- Two-wire half-duplex protocols also exist (RS485, for instance)



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

39

39

USART

Universal Synchronous Asynchronous serial Receiver Transmitter

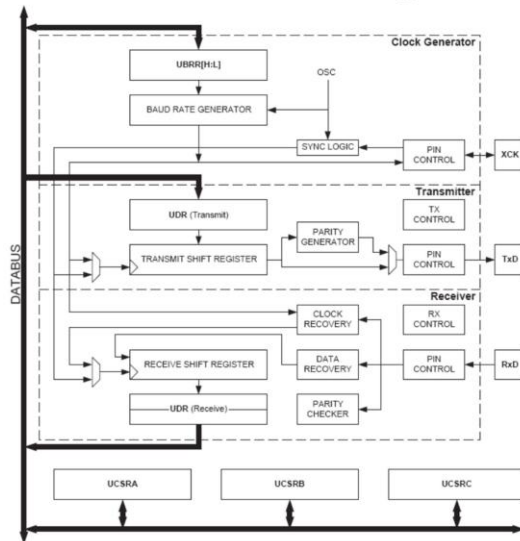
- The ATmega 128 has 2 **USART** peripherals (USART0 e USART1)
- Operation in mode **Full Duplex**
 - independent registers for data transmission and receiving
- **Synchronous** and **Asynchronous** communication modes
- High resolution baud rate generator
- Supports frames with **5, 6, 7, 8 ou 9** data bits and **1 or 2 stop bits**
- Even or odd parity bit generator and hardware parity verification
- Error flags:
 - Data OveRun
 - Framing Error

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

40

40

USART – Block Diagram



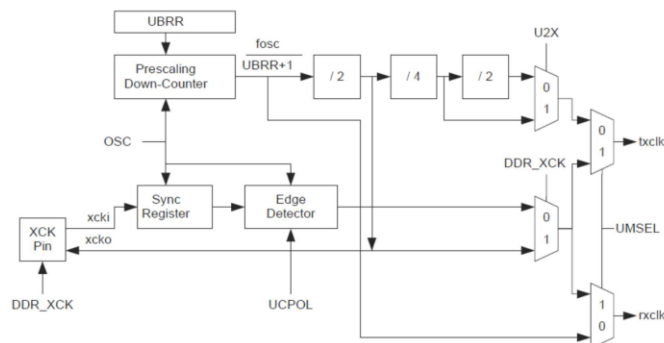
- 3 Main blocks:
 - Clock generator
 - Transmission
 - Reception
- Configuration Registers
 - UCSRA
 - UCSRB
 - UCSRC
- USART Pins
 - XCK (Synchronous)
 - TxD
 - RxD

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

41

41

USART - Baud Rate Generator I



- 4 Operating modes
 - Normal Asynchronous
 - Double Asynchronous speed
 - Synchronous Master
 - Synchronous Slave
- UMSEL Bit selects Synchronous or Asynchronous mode
- UBRR - Baud Rate Register
- Bit U2X selects double speed (asynchronous mode)
- XCK Pin is only active in synchronous mode

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

42

42

USART - Baud Rate Generator II

Operating Mode	Equation for Calculating Baud Rate ⁽¹⁾	Equation for Calculating UBRR Value
Asynchronous Normal Mode (U2X = 0)	$BAUD = \frac{f_{OSC}}{16(UBRR + 1)}$	$UBRR = \frac{f_{OSC}}{16BAUD} - 1$
Asynchronous Double Speed Mode (U2X = 1)	$BAUD = \frac{f_{OSC}}{8(UBRR + 1)}$	$UBRR = \frac{f_{OSC}}{8BAUD} - 1$
Synchronous Master Mode	$BAUD = \frac{f_{OSC}}{2(UBRR + 1)}$	$UBRR = \frac{f_{OSC}}{2BAUD} - 1$

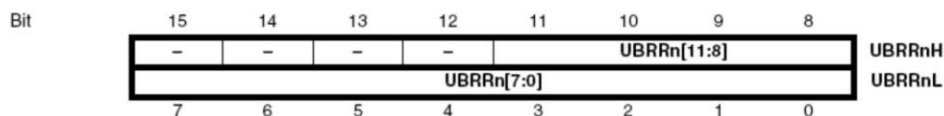
Note: 1. The baud rate is defined to be the transfer rate in bit per second (bps).

Baud Rate (bps)	$f_{osc} = 16.0000 \text{ MHz}$				$f_{osc} = 18.4320 \text{ MHz}$				$f_{osc} = 20.0000 \text{ MHz}$			
	U2X = 0		U2X = 1		U2X = 0		U2X = 1		U2X = 0		U2X = 1	
	UBRR	Error	UBRR	Error	UBRR	Error	UBRR	Error	UBRR	Error	UBRR	Error
2400	416	-0.1%	832	0.0%	479	0.0%	959	0.0%	520	0.0%	1041	0.0%
4800	207	0.2%	416	-0.1%	239	0.0%	479	0.0%	259	0.2%	520	0.0%
9600	103	0.2%	207	0.2%	119	0.0%	239	0.0%	129	0.2%	259	0.2%
14.4k	68	0.6%	138	-0.1%	79	0.0%	159	0.0%	86	-0.2%	173	-0.2%
19.2k	51	0.2%	103	0.2%	59	0.0%	119	0.0%	64	0.2%	129	0.2%
28.8k	34	-0.8%	68	0.6%	39	0.0%	79	0.0%	42	0.9%	86	-0.2%
38.4k	25	0.2%	51	0.2%	29	0.0%	59	0.0%	32	-1.4%	64	0.2%
57.6k	16	2.1%	34	-0.8%	19	0.0%	39	0.0%	21	-1.4%	42	0.9%
76.8k	12	0.2%	25	0.2%	14	0.0%	29	0.0%	15	1.7%	32	-1.4%
115.2k	8	-3.5%	16	2.1%	9	0.0%	19	0.0%	10	-1.4%	21	-1.4%
230.4k	3	8.5%	8	-3.5%	4	0.0%	9	0.0%	4	8.5%	10	-1.4%
250k	3	0.0%	7	0.0%	4	-7.8%	8	2.4%	4	0.0%	9	0.0%
0.5M	1	0.0%	3	0.0%	—	—	4	-7.8%	—	—	4	0.0%
1M	0	0.0%	1	0.0%	—	—	—	—	—	—	—	—
Max ⁽¹⁾	1 Mbps		2 Mbps		1,152 Mbps		2,304 Mbps		1,25 Mbps		2.5 Mbps	

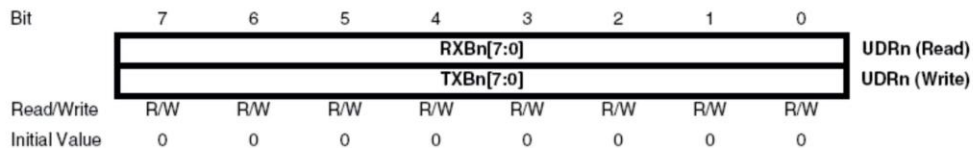
43

43

USART – Registers I



- Baud Rate configuration register (12 bits)



- Registers for data transmission and receiving (UDRn)

44

44

USART – Registers II

Bit	7	6	5	4	3	2	1	0	
	RXCn	TXCn	UDREN	FE_n	DORn	UPEn	U2Xn	MPCMn	UCSRnA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

RXCn Usart Receive Complete (INT)
TXCn Usart Transmit Complete (INT)
UDREN Usart Data Register Empty (INT)
FE_n Frame Error
DORn Data OverRun
UPEn Usart Parity Error
U2Xn Usart double speed
MPCMn Multi Processor Communication Mode

Bit	7	6	5	4	3	2	1	0	
	RXCIE_n	TXCIE_n	UDRIE_n	RXEN_n	TXEN_n	UCSZn2	RXB8n	TXB8n	UCSRnB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

RXCIE_n RX Complete Interrupt Enable
TXCIE_n TX Complete Interrupt Enable
UDRIE_n Usart Data Register empty Interrupt Enable
RXEN_n Receiver Enable
TXEN_n Transmitter Enable
UCSZn2 Character Size
RXB8n Receive Data Bit 8
TXB8n Transmit Data Bit 8

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

45

45

USART – Registers III

Bit	7	6	5	4	3	2	1	0	
	–	UMSEL_n	UPMn1	UPMn0	USBSn	UCSZn1	UCSZn0	UCPOLn	UCSRnC
Read/Write		R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value		0	0	0	0	1	1	0	

UMSEL _n	Modo
0	Assíncrono
1	Síncrono

UPMn1	UPMn0	Paridade
0	0	Sem paridade
0	1	(Reservado)
1	0	Paridade par
1	1	Paridade ímpar

USBSn	Stop bits
0	1 stop bit
1	2 stop bits

UCSZn2	UCSZn1	UCSZn0	Comprimento da palavra
0	0	0	5 bits
0	0	1	6 bits
0	1	0	7 bits
0	1	1	8 bits
1	0	0	(Reservado)
1	0	1	(Reservado)
1	1	0	(Reservado)
1	1	1	9 bits

UCPOLn	Transmissão (novo bit)	Recepção (amostragem do bit)
0	Flanco ascendente de XCKn	Flanco descendente de XCKn
1	Flanco descendente de XCKn	Flanco ascendente de XCKn

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

46

46

EXERCISE

- Write a program that echoes each received character through USART1.
- The USART1 should be configured for 9600 bps, no parity, 8 data bits, 1 stop bit
- Repeat for 4800 bps



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

47

47

SERIAL ECHO

- For 4800 bps:
 - UBRR = 416

```
uint16_t brr = 416;
UBRR1H = (brr >> 8);
UBRR1L = brr & 0xFF;
UCSR1A |= (1<<U2X1);
```

//or simply (416 = 0x1A0)

```
UBRR1H = 1;
UBRR1L = 0xA0;
UCSR1A |= (1<<U2X1);
```

```
int main(void) {
    uint8_t buf;
```

```
//9600 bps
UBRR1H = 0;
UBRR1L = 207;
UCSR1A |= (1<<U2X1);
```

```
UCSR1B |= (1<< RXEN1)|(1<< TXEN1);
UCSR1C |= (1<<UCSZ11)|(1<<UCSZ10);
```

```
while (1) {
    //RXC1 set to 1 when new data on RX buffer
    while( (UCSR1A & (1<<RXC1))==0 );
```

```
    buf = UDR1; //also clears RXC1
```

```
    //UDRE1 set to 1 when TX buffer empty
    while( (UCSR1A & (1<<UDRE1))==0 );
```

```
    UDR1 = buf; //also clears UDRE1
```

```
} }
```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

48

48



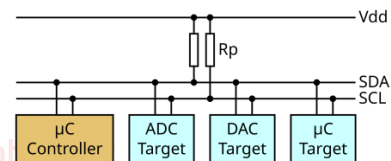
I²C / TWO-WIRE INTERFACE (TWI)

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

49

I²C/TWI: SYNCHRONOUS SERIAL COMMUNICATION

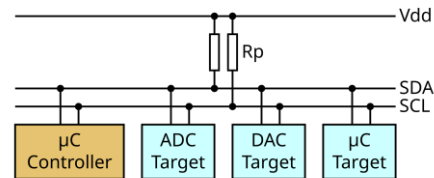
- Allows to connect multiple distinct modules using only 2 wires (Bus)
 - Serial Clock (SCL)
 - Serial Data (SDA)
- Speeds from 100 kbit/s up to 5 Mbit/s (depending on the devices)
- Originally designed to connect ICs on the same PCB
 - But distances of a few meters are also possible, with decreasing bit rates.



50

I²C/TWI: SYNCHRONOUS SERIAL COMMUNICATION

- Controller/target (master/slave) network where each module must have its own address
 - Half-duplex communications
 - Theoretical maximum of 127 devices on the same bus (standard 7-bit addressing)
 - 10-bit addresses can also be used (not widely used)
 - In practice, due to load capacitance, number might have to be much lower
 - But a device can act as gateway for other bus
- 2 pull-up resistors, one for each bus line
- Multiple controllers allowed. Arbitration:
 - Open drain outputs act as an "Wired-AND" (if one device writes 0 and other 1, the bus stays on 0)
 - In the case of the Atmega, internal pull-ups may be used

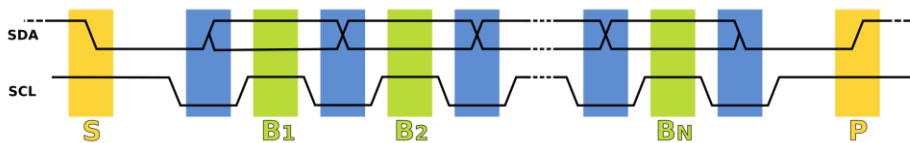


PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

51

51

I²C/TWI: SYNCHRONOUS SERIAL COMMUNICATION



- One bit transmission (SDA) for each clock pulse(SCL)
 - Transactions are delimited by start (S) and stop bit (P)
 - These are special bits: SDA's state is changed when SCL is 1
 - For all other bits ($B_1, \dots, B_N$), data line (SDA) must be stable when SCL clock line is 1
- Transmissions consist of several packets and different types of transactions

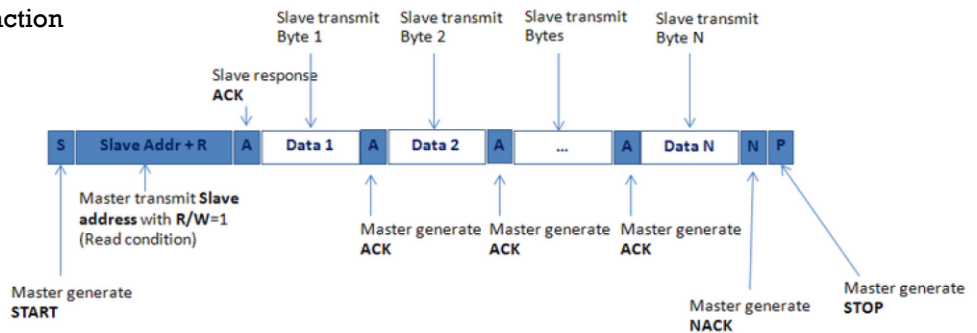
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

52

52

I2C TRANSMISSIONS

Read transaction



Write transaction

Combined transaction

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

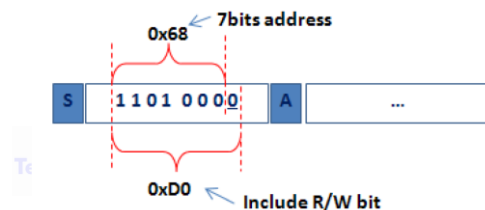
53

53

I²C PACKET FORMATS (FOR 7-BIT ADDRESSES)

Address Packets

- 7 address bits
- 1 control bit (Read/Write)
- 1 acknowledge bit



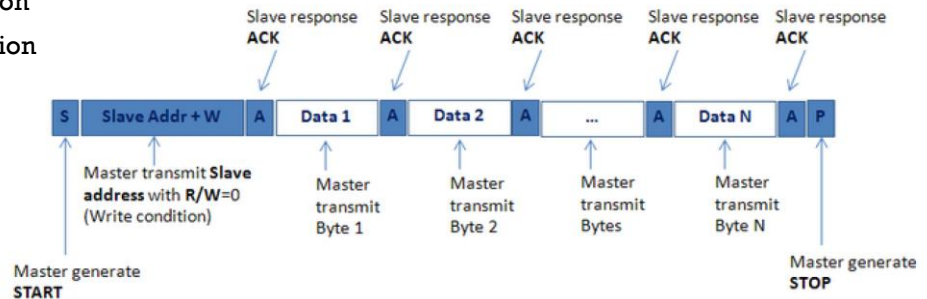
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

54

54

I²C TRANSMISSIONS

- Read transaction
- Write transaction



- Combined transaction

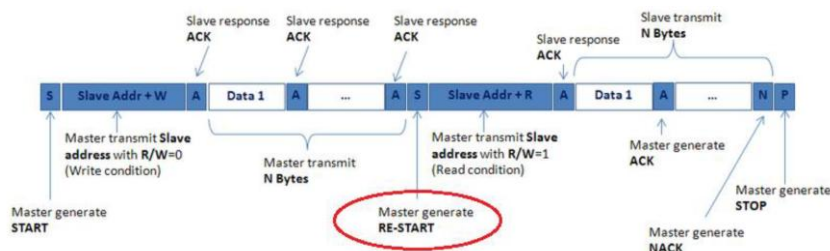
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

55

55

I²C TRANSMISSIONS

- Combined transaction – Used to send “arguments” to the slave, before reading the corresponding reply
 - Start Communication (Start bit), send write request, but **do not** send **Stop bit**
 - Used, for instance, to specify the address of a register on the destination device
 - Repeat the Start bit and send a read request
 - Terminate the transaction by sending the Stop bit



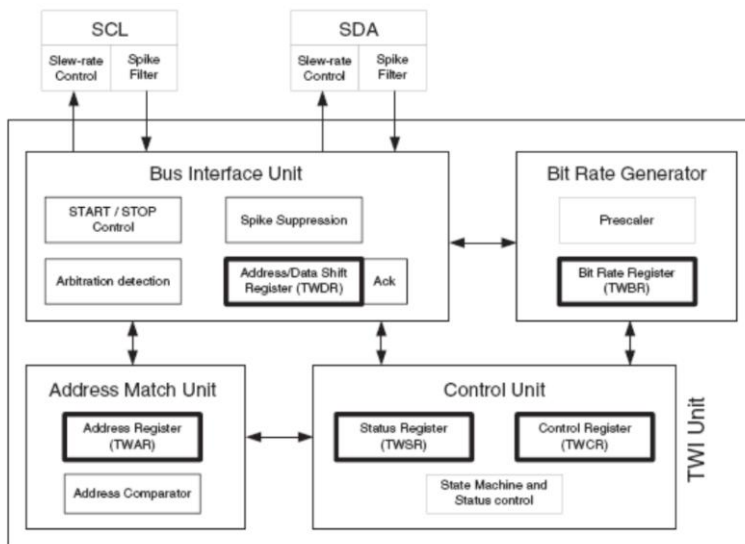
56

56

I²C BROADCASTS

- Address 0 is reserved for “general calls” (broadcasts)
 - To send the same message to all devices
 - For special programming and reset operations

57



ATMEGA128 TWO-WIRE SERIAL INTERFACE

- Control Register (TWCR)
- Status Register (TWSR)
- Bit rate Register (TWBR)
- Data Register (TWDR)
 - Also used to specify target address
- Address Register (TWAR)
 - Only used in target mode

58

TWI – Bit Rate Generator

- This block controls the clock period (**SCL**) when the TWI is in **Master mode**

- The frequency value is calculated using the equation:

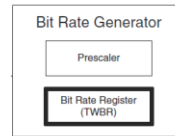
$$\text{SCL frequency} = \frac{\text{CPU Clock frequency}}{16 + 2(\text{TWBR}) \cdot 4^{\text{TWPS}}}$$

Standard-mode -> 100 kbit/s

Fast-mode -> 400 kbit/s

- TWBR = TWI Bit Rate Register
- TWPS = prescaler bits of TWI Status Register (0,1,2 e 3)
- TWBR – TWI Bit Rate Register

Bit	7	6	5	4	3	2	1	0	
	TWBR7	TWBR6	TWBR5	TWBR4	TWBR3	TWBR2	TWBR1	TWBR0	TWBR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	



TWPS1	TWPS0	Prescaler Value
0	0	1
0	1	4
1	0	16
1	1	64

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

59

59

EXEMPLO

- ATmega128@16 MHz
- Transmission rate (TWI_FREQ) of 100 kbit/s
- With no prescaler (TWPS = 0):
 - $\text{TWBR} = ((F_{\text{CPU}} / \text{TWI_FREQ}) - 16) / 2 = 72$

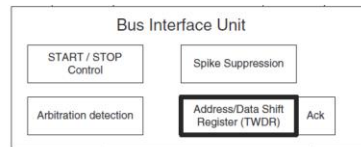


PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

60

60

TWI – Bus Interface Unit



- This block consists of the TWDR (TWI Data Register) register containing the **address** or **data** to be transmitted or received.

Bit	7	6	5	4	3	2	1	0	
(0xBB)	TWD7	TWD6	TWD5	TWD4	TWD3	TWD2	TWD1	TWD0	TWDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	1	1	1	1	1	1	1	1	

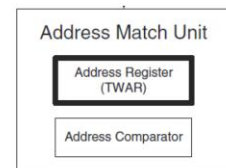
- In this block, the following functions are also performed:
 - START and STOP bit control
 - Media access control when there is more than one master.
 - Bit control (N)ACK

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

61

61

TWI – Address Match Unit



- This block compares the received address and the address (7 bits) stored in the TWAR register (*TWI (Slave) Address Register*).
- An address is only required if the system works as a **slave**

Bit	7	6	5	4	3	2	1	0	
(0xBA)	TWA6	TWA5	TWA4	TWA3	TWA2	TWA1	TWA0	TWGCE	TWAR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	1	1	1	1	1	1	1	0	

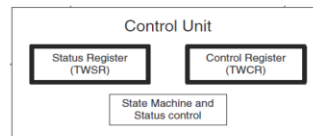
- Bit TWGCE – enable comparison with generall call address (0x00)

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

62

62

TWI – Control Unit I



- This block monitors the TWI bus and generates alerts according to settings made on the TWI *Control Register* (TWCR)

Bit (0xBC)	7	6	5	4	3	2	1	0	
Read/Write	R/W	R/W	R/W	R/W	R	R/W	R	R/W	TWCR
Initial Value	0	0	0	0	0	0	0	0	

TWINT	TWI Interrupt Flag
TWEA	TWI Enable Acknowledge Bit
TWSTA	TWI START Condition Bit
TWSTO	TWI STOP Condition Bit
TWWC	TWI Write Collision Flag
TWEN	TWI Enable Bit
TWIE	TWI Interrupt Enable

- TWINT flag is set to 1 by the hardware whenever an event on the TWI bus ends.

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



63

TWI – Control Unit II

- The TWINT flag is set to 1 by the hardware whenever an event on the TWI bus ends, such as:
 - after transmitting start/repeated start
 - after transmitting SLA+R/W
 - after transmitting data byte
 - after receiving data byte
 - Transmission errors
- After completing an event the TWSR register is updated with event information.
- While TWINT is 1 the SCL pin is 0 -> allows the application to perform the necessary procedures.
- TWINT** flag must be cleared by software -> write logical value 1.

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



64

TWI – Control Unit III

- TWSR (TW Status Register) register stores information (TWST 7: 3) about the event that triggered TWINT flag.

Bit	7	6	5	4	3	2	1	0	
(0xB9)	TWS7	TWS6	TWS5	TWS4	TWS3	–	TWPS1	TWPS0	TWSR
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial Value	1	1	1	1	1	0	0	0	

- 4 operating modes

- Master Transmitter
- Master Receiver
- Slave Transmitter
- Slave Receiver

TWPS1 ... 0 TWI Prescaler Bits

TWPS1	TWPS0	Prescaler
0	0	1
0	1	4
1	0	16
1	1	64

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

65

65

TWI – Control Unit IV

TWSR information when the system is in **Master Receiver** mode

Status Code (TWSR) Prescaler Bits are 0	Status of the Two-wire Serial Bus and Two-wire Serial Interface Hardware	Application Software Response					Next Action Taken by TWI Hardware
		To/from TWDR	To TWCR				
			STA	STO	TWINT	TWEA	
\$08	A START condition has been transmitted	Load SLA+R	0	0	1	X	SLA+R will be transmitted ACK or NOT ACK will be received
\$10	A repeated START condition has been transmitted	Load SLA+R or	0	0	1	X	SLA+R will be transmitted ACK or NOT ACK will be received SLA+W will be transmitted Logic will switch to master transmitter mode
		Load SLA+W	0	0	1	X	
\$38	Arbitration lost in SLA+R or NOT ACK bit	No TWDR action or	0	0	1	X	Two-wire Serial Bus will be released and not addressed slave mode will be entered A START condition will be transmitted when the bus becomes free
		No TWDR action	1	0	1	X	
\$40	SLA+R has been transmitted; ACK has been received	No TWDR action or	0	0	1	0	Data byte will be received and NOT ACK will be returned Data byte will be received and ACK will be returned
		No TWDR action	0	0	1	1	
\$48	SLA+R has been transmitted; NOT ACK has been received	No TWDR action or	1	0	1	X	Repeated START will be transmitted STOP condition will be transmitted and TWSTO flag will be reset STOP condition followed by a START condition will be transmitted and TWSTO flag will be reset
		No TWDR action or	0	1	1	X	
		No TWDR action	1	1	1	X	
\$50	Data byte has been received; ACK has been returned	Read data byte or	0	0	1	0	Data byte will be received and NOT ACK will be returned Data byte will be received and ACK will be returned
		Read data byte	0	0	1	1	
\$58	Data byte has been received; NOT ACK has been returned	Read data byte or	1	0	1	X	Repeated START will be transmitted STOP condition will be transmitted and TWSTO flag will be reset STOP condition followed by a START condition will be transmitted and TWSTO flag will be reset
		Read data byte or	0	1	1	X	
		Read data byte	1	1	1	X	

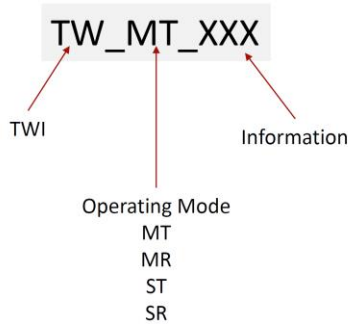
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

66

66

Library twi.h

Mnemonic Interpretation



```

<util/twi.h>: TWI bit mask definitions -

TWSR values
Mnemonics:
TW_MT_XXX - master transmitter
TW_MR_XXX - master receiver
TW_ST_XXX - slave transmitter
TW_SR_XXX - slave receiver
#define TW_START 0x08
#define TW_REP_START 0x10
#define TW_MT_SLA_ACK 0x18
#define TW_MT_SLA_NACK 0x20
#define TW_MR_DATA_ACK 0x28
#define TW_MT_DATA_NACK 0x30
#define TW_MT_ARB_LOST 0x38
#define TW_MR_ARB_LOST 0x38
#define TW_MR_SLA_ACK 0x40
#define TW_MR_SLA_NACK 0x48
#define TW_MR_DATA_NACK 0x50
#define TW_MR_DATA_ACK 0x58
#define TW_ST_SLA_ACK 0xA8
#define TW_ST_ARB_LOST_SLA_ACK 0xB0
#define TW_ST_DATA_ACK 0xB8
#define TW_ST_DATA_NACK 0xC0
#define TW_ST_LAST_DATA 0xC8
#define TW_SR_SLA_ACK 0x60
#define TW_SR_ARB_LOST_SLA_ACK 0x68
#define TW_SR_GCALL_ACK 0x70
#define TW_SR_ARB_LOST_GCALL_ACK 0x78
#define TW_SR_DATA_ACK 0x80
#define TW_SR_DATA_NACK 0x88
#define TW_SR_GCALL_DATA_ACK 0x90
#define TW_SR_GCALL_DATA_NACK 0x98
#define TW_SR_STOP 0xA0
#define TW_NO_INFO 0xF8
#define TW_BUS_ERROR 0x00
#define TW_STATUS_MASK (TWSR & TW_STATUS_MASK)

R/~W bit in SLA+R/W address field.
#define TW_READ 1
#define TW_WRITE 0
  
```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

67

67

Master Transmitter

Write 0x00 in register 0x6B

```

TWSR = (1<<TWINT)|(1<<TWSTA)|(1<<TWEN);

while (!(TWSR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_START)
    erro();

TWR = 0x00;
TWSR = (1<<TWINT)|(1<<TWEN);

while (!(TWSR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_MT_SLA_ACK)
    erro();

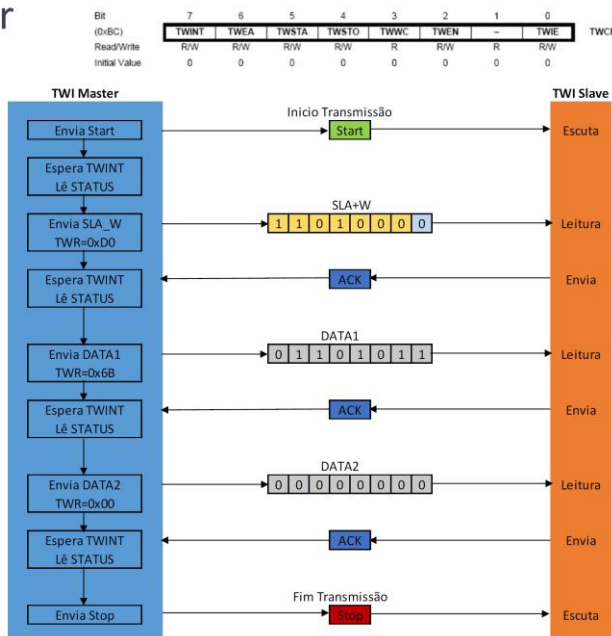
TWR = 0x6B;
TWSR = (1<<TWINT)|(1<<TWEN);

while (!(TWSR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_MT_DATA_ACK)
    erro();

TWR = 0x00;
TWSR = (1<<TWINT)|(1<<TWEN);

while (!(TWSR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_MT_DATA_ACK)
    erro();

TWSR = (1<<TWINT)|(1<<TWSTO)|(1<<TWEN);
  
```



68

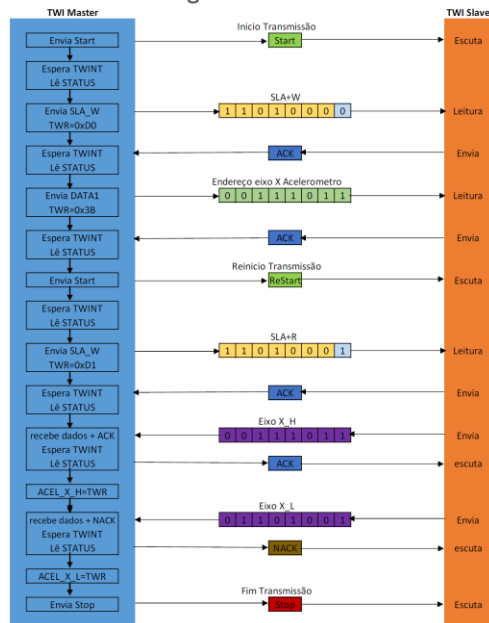
68

Master Transmitter/receiver - Reading accelerometer X-axis

```

TWCR = (1<<TWINT)|(1<<TWSTA)|(1<<TWEN);
while (!(TWCR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_START)
    erro();
    TWDR = 0xD0;
    TWCR = (1<<TWINT)|(1<<TWEN);
while (!(TWCR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_MT_SLA_ACK)
    erro();
TWDR = 0x3B;
TWCR = (1<<TWINT)|(1<<TWEN);
while (!(TWCR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_MT_DATA_ACK)
    erro();
TWCR = (1<<TWINT)|(1<<TWSTA)|(1<<TWEN);
while (!(TWCR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_REP_START)
    erro();
    TWDR = 0xD1;
    TWCR = (1<<TWINT)|(1<<TWEN);
while (!(TWCR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_MR_SLA_ACK)
    erro();
TWCR = (1<<TWINT)|(1<<TWEN)|(1<<TWIEA);
while (!(TWCR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_MR_DATA_ACK)
    erro();
    ACCEL_X_H = TWDR;
    TWCR = (1<<TWINT)|(1<<TWEN);
while (!(TWCR & (1<<TWINT)));
if((TWSR & 0xF8) != TW_MR_DATA_NACK)
    erro();
    ACCEL_X_L = TWDR;
    TWCR = (1<<TWINT)|(1<<TWSTO)|(1<<TWEN);

```



69

69



ACKNOWLEDGEMENTS

POSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

70

ACKNOWLEDGEMENTS

- These slides are partially based on the notes from the **Microprocessors and Microcontrollers** course unit of the **Electrical and Computer Engineering** degree from **Instituto Superior de Engenharia do Porto**, with some of them being direct transcriptions of slides from that course unit.
- Several diagrams and tables from the ATmega128 datasheet are used on these slides:
 - <https://www1.microchip.com/downloads/en/DeviceDoc/doc2467.pdf>

